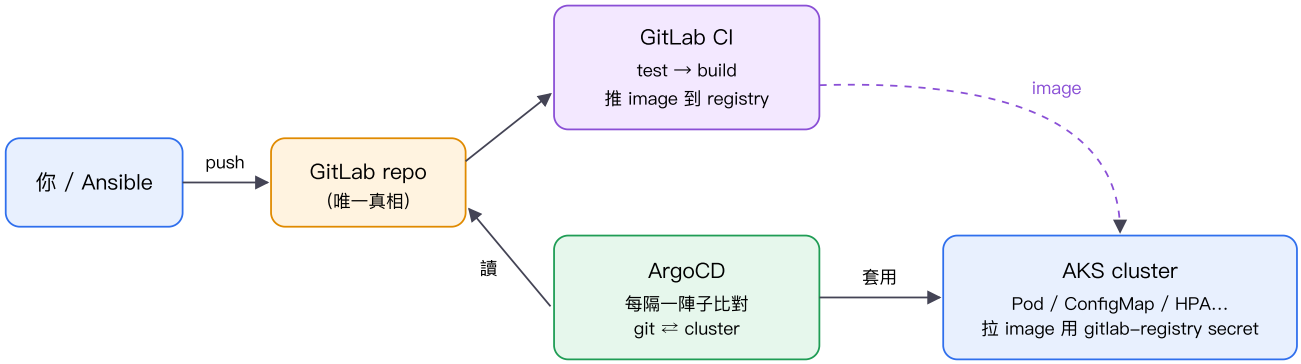
Day 16–18 圖解:GitOps 收尾 + Ansible

今天做了什麼、為什麼這樣做、面試能怎麼講(含誠實限制)

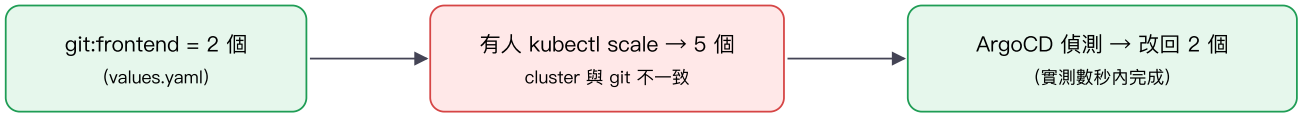
一句話總結:現在「改程式 → 上線」和「改設定 → 上線」都只靠 `git push`,沒有人手動下 kubectl / helm。Day 16 補上自我修復示範,Day 17 學冪等性,Day 18 用 Ansible 把「安全地改設定」自動化。

1. 全貌:今天這條線在整個專案的位置



關鍵設計:**人只碰 git**。ArgoCD 在 cluster 裡把 git 的內容變成現實;有人手動改 cluster,ArgoCD 會改回去。

2. Day 16:selfHeal 自我修復示範



學到什麼: `syncPolicy.automated.selfHeal: true` 讓 git 成為唯一真相。`prune: true` 則是 git 刪掉的資源 cluster 也跟著刪。

前面踩過的坑(別放進面試故事,但要懂):叢集是 ARM 節點,CI 一開始建的是 amd64 image,所以後來改成在 arm64 runner 上建;GitLab 的 `CI_COMMIT_SHORT_SHA` 是 8 碼,tag 打成 7 碼會拉不到 image。

3. Day 17:冪等性(idempotent)

定義:同一份 playbook 跑 1 次或 10 次,結果一樣;已經是目標狀態就不再動。Ansible 的模組是「宣告目標狀態」,不是「執行指令」。

寫法	意思	第 2 次跑
`file: state=directory`	目錄要存在	ok(不動)
`copy: content=…`	檔案內容要是這樣	ok
`lineinfile: line=…`	這一行要存在	ok
`shell: echo … >> log`	只是執行指令	**changed**(每次都多一行)



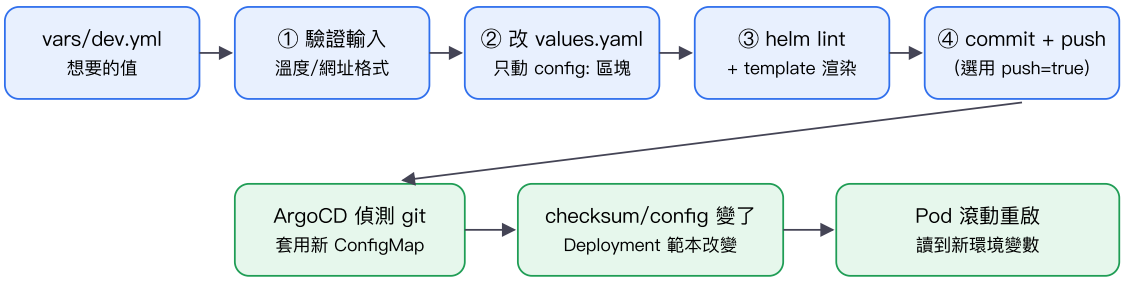
我自己踩到的:demo 第一版,`copy` 和 `lineinfile` 管同一個檔案,互相覆蓋,第二次跑 changed=3。教訓:**兩個 task 不要管同一份狀態**,否則永遠不穩定。分開檔案後才變成預期的 changed=1。

誠實限制:沒有遠端 VM,我沒做「在遠端裝 docker」,只在本機 localhost 示範冪等概念。

4. Day 18:用 Ansible 更新 ConfigMap(走 GitOps)

為什麼 playbook 改 git,而不是直接 kubectl apply ConfigMap?

因為 ArgoCD 有 selfHeal:直接改 cluster 的 ConfigMap,幾秒就被改回 git 的版本。所以設定變更必須走 git,playbook 自動化的是「**安全地改 git**」那一段。



實測	結果
溫度填 9(壞輸入)	assert 擋下,**沒寫任何東西**
`--check --diff`	只預覽 `- 0.6 / + 0.3`,不改檔
正式執行	changed=1,再跑一次 changed=0(冪等)
push 後	ArgoCD 套用,新 backend Pod 內 `TEMPERATURE=0.3`

誠實限制:溫度對 `gemini-3.6-flash` 其實不生效(模型忽略),這裡只是用它示範「設定變更流程」。被問到就直說。另外 push 當下 ArgoCD 短暫顯示 Degraded,是滾動更新中的暫時狀態,幾十秒後回到 Healthy。

5. 面試:可能被問的問題與回答方向

Q1. GitOps 是什麼?跟傳統 CI 直接 kubectl apply 差在哪?

期望狀態寫在 git,cluster 裡有 agent(ArgoCD)持續比對並收斂。差別:① cluster 憑證不必放在 CI;② 每個變更都有 git 紀錄可審計、可 revert 回滾;③ 手動漂移會被自動修正。

Q2. selfHeal 和 prune 各是什麼?什麼情況你會關掉 selfHeal?

selfHeal:cluster 被手動改了就改回 git。prune:git 刪掉的資源 cluster 也刪。緊急排障要臨時手動調整時可先關 selfHeal,事後再把修正寫回 git。

Q3. 為什麼用 Ansible 改 git 而不是直接 apply ConfigMap?

因為有 selfHeal,直接改會被還原;而且走 git 保留審計軌跡。playbook 還加了輸入驗證與 lint,取代手動編輯 YAML 易打錯的重複工作。

Q4. 冪等性是什麼?你怎麼確保?

跑一次與跑多次結果相同。用宣告式模組(file/copy/lineinfile)而非 shell;shell 要用 `creates` / `changed_when`。驗證方式:連跑兩次,第二次 `changed=0`。可講我踩過「兩個 task 管同一檔案互相覆蓋」的 bug。

Q5. ConfigMap 改了,Pod 為什麼會自動重啟?

環境變數只在啟動時讀一次。chart 在 Pod 範本放 `checksum/config` annotation(ConfigMap 內容的雜湊),內容一變,範本變 → Deployment 滾動更新。

Q6. 私有 registry 的 image,Pod 怎麼拉?

用 deploy token(`read_registry`)建 `docker-registry` secret,在 chart 以 `imagePullSecrets` 引用。secret 含 token,**不進 git**,由人手動建立。

Q7. 遇過什麼問題?(STAR 素材)

image 架構不符:叢集是 ARM、CI 建 amd64,Pod 起不來。處理:改用 arm64 runner 建置,並確認 tag 對應 8 碼 SHA。學到:CI 產物架構要與目標節點一致。

Q8. 這套東西有哪些不足?(主動講加分)

- Ansible 只示範在本機,沒管理過遠端機器。
- image tag 仍要手動(或腳本)寫回 values.yaml,尚未做到「CI 自動更新 tag」。
- pull secret、ArgoCD repo 憑證是手動建立,沒進 GitOps(需 Sealed Secrets / External Secrets)。
- Prometheus 無持久化,重啟歷史消失(見 SLO 文件)。

6. 現況與下一步

天	狀態
Day 14 CI	完成(test→build,壞測試擋住 build 已演示)
Day 15–16 ArgoCD / GitOps	完成(Synced/Healthy、selfHeal 實測)
Day 17–18 Ansible	完成(本機)
Day 19 Istio	建議跳過(優先序最低、單小節點吃緊)

Day 20 Redis	**下一步**:修掉 Day 9 的多 replica 記憶 bug

本文件依 2026-11-01 的實測整理;AKS 已停止,最新 commit 9ea7f7f。